

EASYHARSHMODS PRESENTS

let's build!

20 Days of AI Engineering

From DevOps Engineer → AI Engineer

A hands-on series for cloud, DevOps & platform engineers. Build local AI apps, RAG pipelines, and production services — Python · Ollama · Streamlit · FastAPI · AWS.

● FOUNDATIONS ● LOCAL AI APPS ● RETRIEVAL & RAG ● SHIP IT



FOUNDATIONS

Python Setup for AI Engineering

WHAT YOU BUILD

Install Python 3.14 with pyenv, create an isolated virtual environment, and run a Hello World project that prints your environment status.

WHY IT MATTERS

Every later lesson runs in its own venv. Get setup right once and you can jump to any day and run it standalone.

pyenv

venv

interpreter

project layout



FOUNDATIONS

Python Basics for Cloud Engineers

lists

dicts

loops

functions

type hints

WHAT YOU BUILD

Process AWS-like account data: total monthly cost, accounts over budget, and accounts carrying risk flags — using lists, dicts, loops and typed functions.

WHY IT MATTERS

AI apps still need ordinary data prep before prompts, API responses, and retrieval. These are the building blocks.



FOUNDATIONS

Files, JSON & Config

What you build

Read accounts and a config file, compute a report, and write it back out as structured JSON.

Why it matters

Config-driven scripts stay flexible. JSON is the lingua franca of APIs, model payloads, and knowledge bases.

open / with

`json.load`

`json.dump`

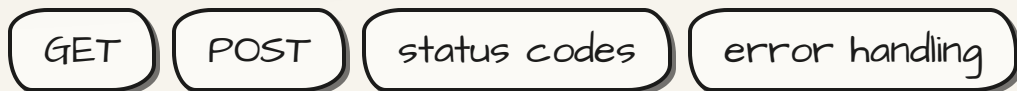
config files

DAY
04

FOUNDATIONS

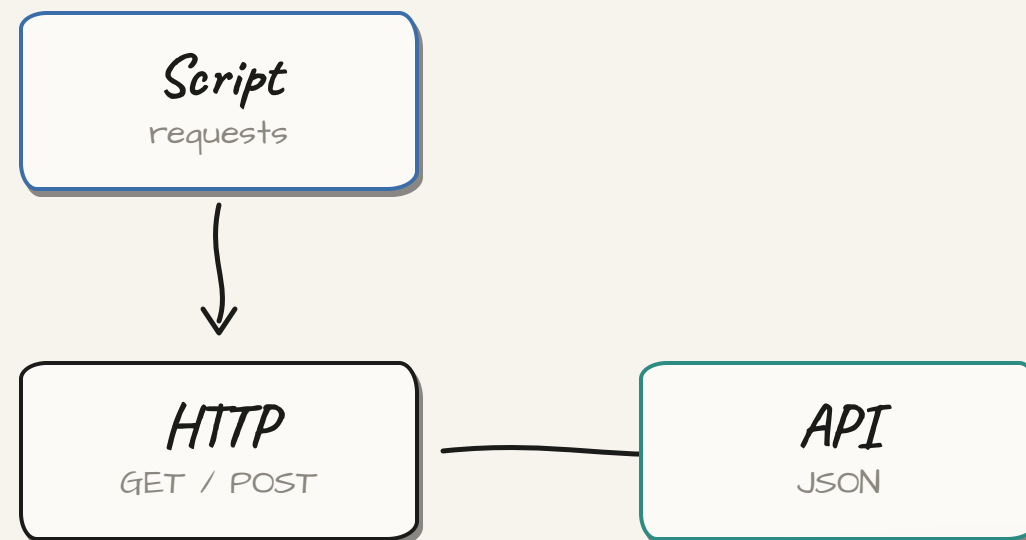
API Calls with Python

✎ ARCHITECTURE



Call a public HTTP API with requests, handle status codes and failures gracefully, print structured output.

WHY: Calling an LLM is just an HTTP call. Master requests now and Day 5's Ollama integration is trivial.



An LLM call is the same request → response loop.

DAY
05

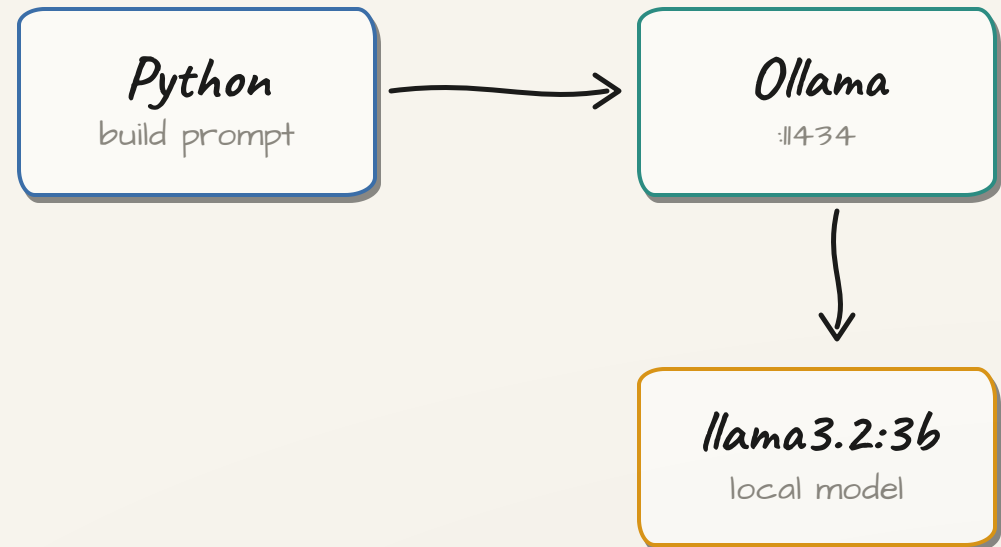
LOCAL AI APPS

Local LLM with Ollama ARCHITECTURE

Runs entirely on your machine - no keys, no limits.



Send a prompt to a local model through Ollama's localhost API and print the answer — no cloud, no API keys.



WHY: Local models mean free iteration, full privacy, no rate limits. This is your AI runtime for the whole series.



LOCAL AI APPS

Modular Python Project

main.py

modules

imports

separation

WHAT YOU BUILD

Refactor the LLM call into a reusable client with config, prompts, a clean entry point — plus graceful error handling.

WHY IT MATTERS

The LocalLlmError pattern you build here is reused by every app from now on. Clean boundaries make code teachable.



LOCAL AI APPS

Streamlit AI App

Streamlit

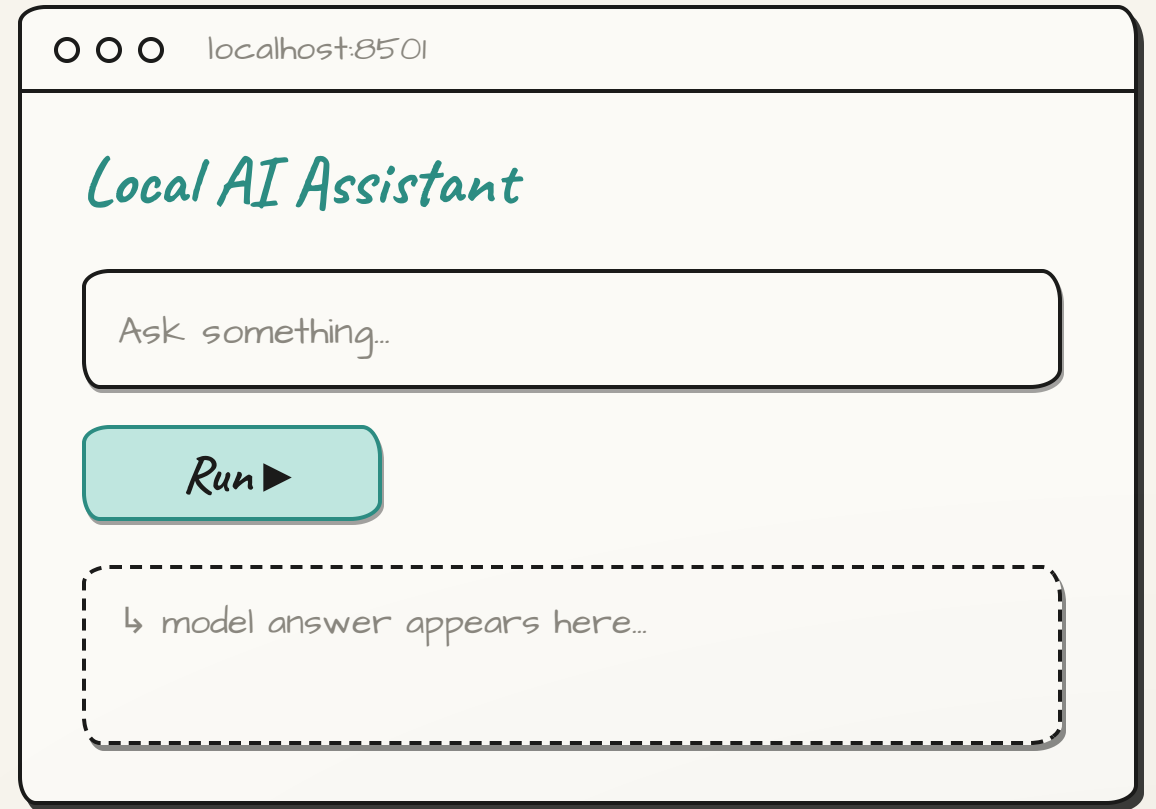
inputs

buttons

model call

Wrap the local model in a simple web UI: a text box, a model selector, and a button that returns the answer in the browser.

WHY: A UI turns a script into a demo. Streamlit gets you there in minutes, no frontend stack required.





LOCAL AI APPS

Chat UI & Session Memory

chat_input

chat_message

session_state

Add a chat interface with conversation history kept in session state, so the assistant remembers earlier turns.

WHY: Memory is the difference between a one-shot prompt and a conversation. Session state is the simplest honest version.

How do I roll back the API?

First confirm the active version, then...

And if errors spike after?

Watch error rate & latency, then...

session_state remembers the whole thread ↑



LOCAL AI APPS

File Upload AI Assistant

What you build

Let users upload a text file, extract its content, build a prompt around it, and answer questions about the document.

Why it matters

Grounding answers in a user's own file is the first taste of retrieval — the motivation for the RAG block.

file upload

text extraction

document Q&A



LOCAL AI APPS

PDF Summarizer

pypdf

extraction

summarization

WHAT YOU BUILD

Extract text from an uploaded PDF with pypdf, preview it, and produce a local summary with the model.

WHY IT MATTERS

PDFs are where real-world knowledge hides — runbooks, contracts, manuals. Extracting them cleanly is core.



RETRIEVAL & RAG

Prompt Engineering for Engineers

Build three reusable prompt templates — AWS explainer, Terraform reviewer, incident summarizer — behind one mode dispatcher.

*AWS
Explainer*

*Terraform
Reviewer*

*Incident
Summarizer*

→ one `build_prompt` dispatcher routes between modes



RETRIEVAL & RAG

Local Document Assistant

text + PDF

prompt modes

bounded context

Combine chat, text/PDF upload, and prompt modes into one Streamlit app that sends a bounded slice of the document to the model.

WHY: A complete local AI workflow — and it hits the wall of prompt-stuffing, which is exactly why RAG comes next.



DAY
13

RETRIEVAL & RAG

Introduction to RAG

📁 ARCHITECTURE

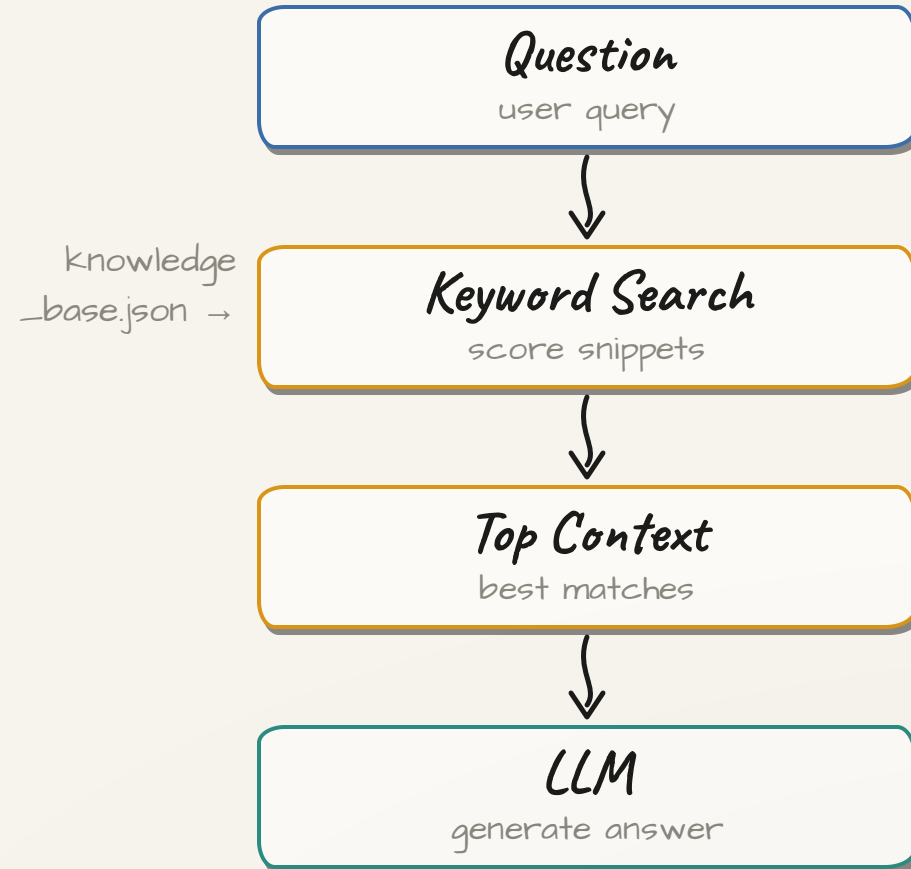
retrieval

generation

keyword scoring

Build the smallest honest RAG: naive keyword scoring over JSON snippets feeds the most relevant context into the model.

WHY: Retrieve-then-generate is the architecture behind most document AI. Building it crudely first makes upgrades obvious.





RETRIEVAL & RAG

Text Chunking

chunk size

overlap

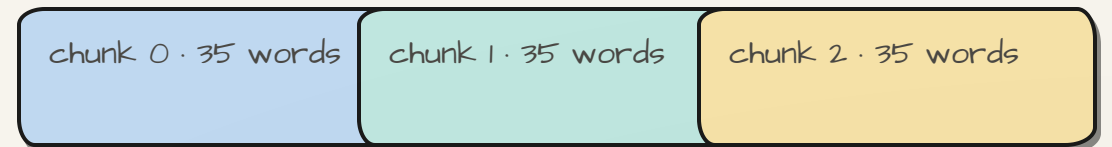
metadata

WHAT YOU BUILD

Split documents into overlapping word chunks with source metadata so retrieval can search small, useful pieces.

WHY: Chunking quietly decides retrieval quality. Too small loses context; too large buries the answer. A tunable choice.

one long document



↳ chunks overlap so no sentence is lost at the seam

DAY
15

RETRIEVAL & RAG

Embeddings Explained

📌 ARCHITECTURE

nomic-embed-text

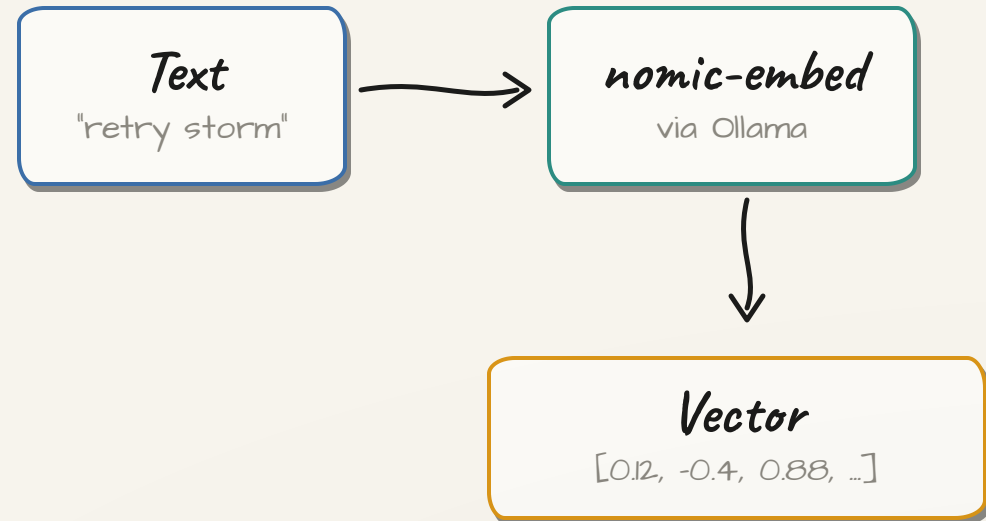
vectors

cosine similarity

Turn text into vectors with nomic-embed-text via Ollama and rank notes by cosine similarity to a query.

WHY: Embeddings let retrieval match meaning, not spelling — the fix for the keyword misses you saw on Day 13.

Similar meaning → similar vectors. Rank by cosine.





RETRIEVAL & RAG

Vector Search

top-k

similarity ranking

numpy

WHAT YOU BUILD

Store chunk vectors in memory, embed a query, and return the top-k matches with scores and source metadata.

WHY: Vector search is the retrieval core of RAG. Building it in numpy first shows what a real vector DB buys you.

top-k by similarity

0.80

retry-runbook · backoff & jitter

0.41

rollback-runbook · prev version

0.36

cost-runbook · idle compute

↑ the retry chunk ranks first for a retry question



RETRIEVAL & RAG

Local RAG Assistant

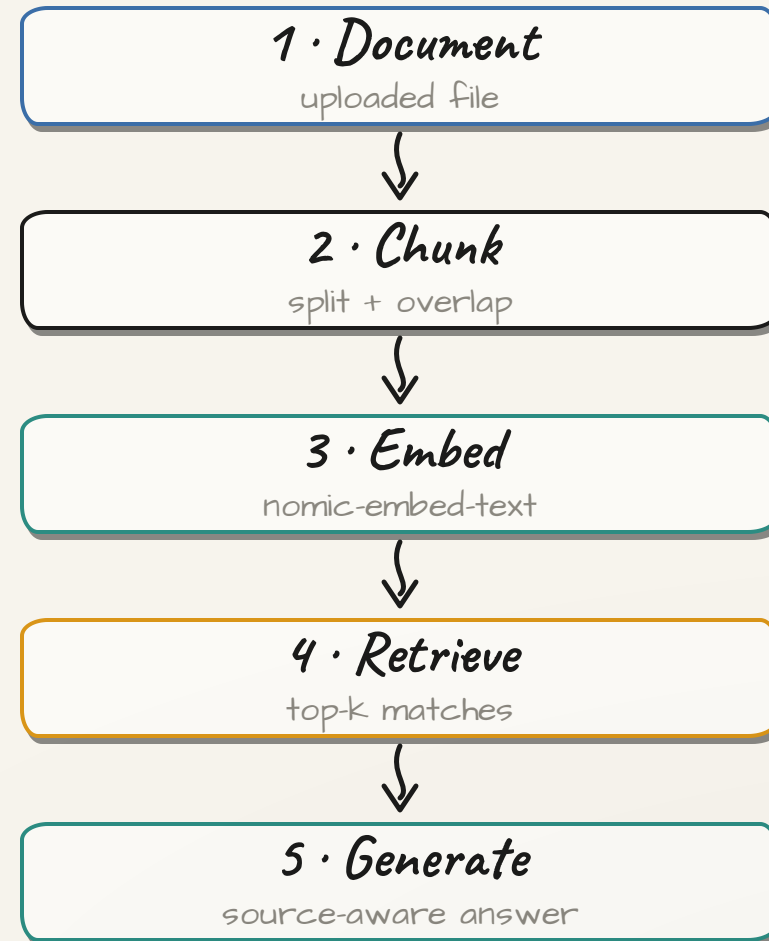
chunk → embed → retrieve → generate

sources

Assemble the full pipeline in Streamlit: chunk an uploaded doc, embed it, retrieve top-k, generate a source-aware answer.

WHY: The payoff of the RAG block — the same architecture behind most document AI products, small enough to read end to end.

📌 ARCHITECTURE





SHIP IT

FastAPI Wrapper

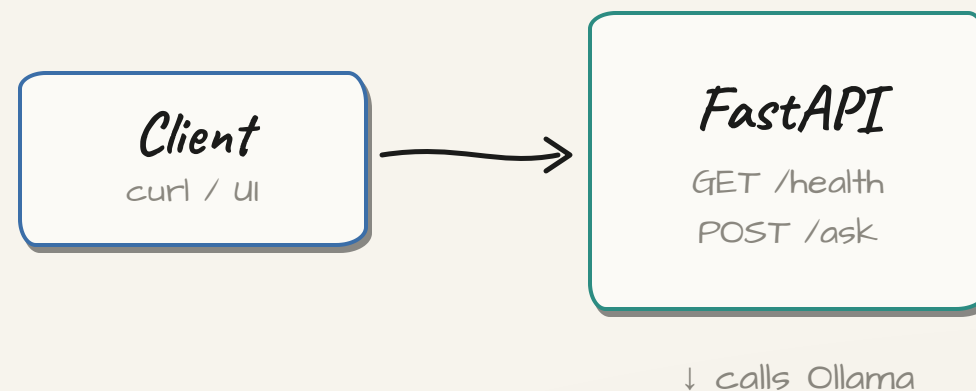


Expose the assistant as a REST API with a health check and a typed /ask endpoint, plus automatic interactive docs.

WHY: A script helps one person; an API helps every system that can send a request. The step from tool to service.

📌 ARCHITECTURE

Typed models + auto docs at /docs.





SHIP IT

Dockerize the AI App

Dockerfile

ports

container runtime

WHAT YOU BUILD

Containerize the FastAPI service with a slim Python image, keeping Ollama on the host so the image stays focused.

WHY: Containers make the service run identically on a laptop, in CI, and in the cloud — the foundation for AWS.



container

FastAPI

code.api:app

EXPOSE 8000

Ollama

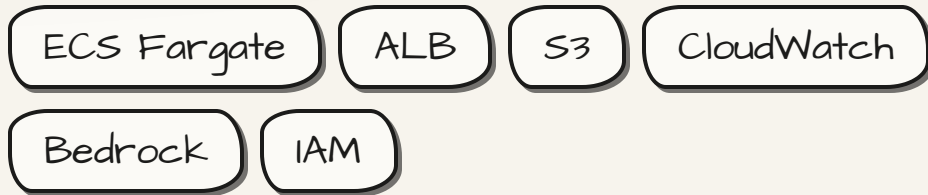
on the host

model stays outside the image

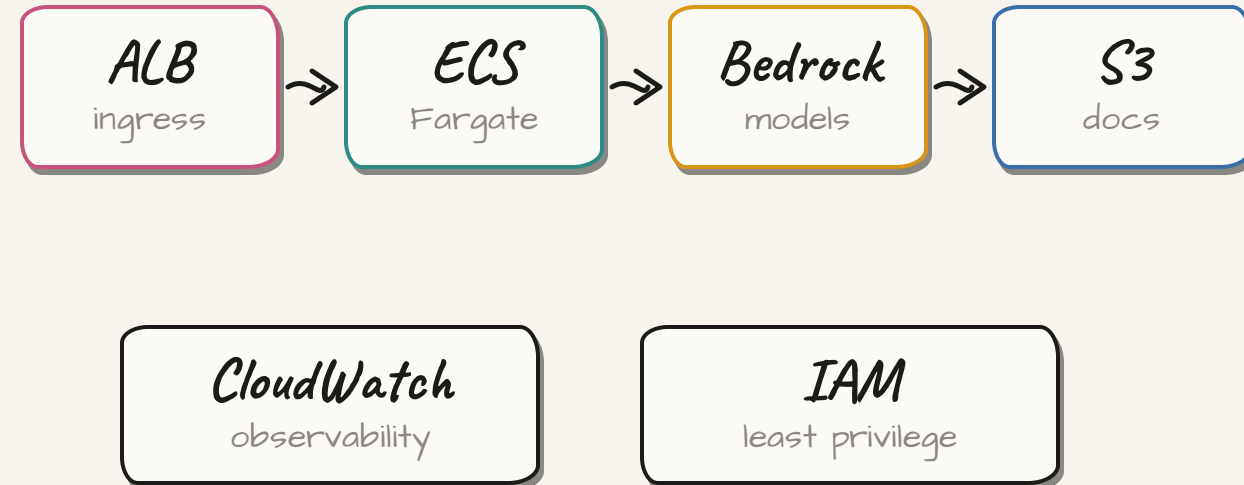
DAY
20

SHIP IT

AI Engineering on AWS ARCHITECTURE



Map the local project to a production AWS architecture and review a least-privilege IAM policy and deployment skeleton.



Local script → production AWS service.

WHY: Connects the whole series to how AI services actually run in production — security, cost, and scale included.

by Day 20, you've shipped

a real AI service.

Local-first

Everything runs on your machine with Ollama — no keys, no bills while you learn.

Production-minded

Every lesson ends with what changes in production: security, cost, scale.

Independent

Jump to any day. Each folder runs standalone in its own venv.

Script → Cloud

Python basics → RAG → FastAPI → Docker → AWS architecture.

Start at Day 01 → github.com/easyharshmods